مقدمه و راهنمای استفاده

به نام تک‌برنامه‌نویس هستی؛ من مهشید شیدایی هستم و این مجموعه را برای شما آماده کرده‌ام تا با حل مسئله واقعی، پایتون را نه فقط حفظ کنید بلکه با آن زندگی کنید.

این کتاب ۱۰۰ تمرین مستقل و اجراپذیر دارد که در ۱۰ بخش موضوعی سازماندهی شده‌اند: از شرط‌های ساده و حلقه‌ها تا پردازش متن، ساختارهای داده، توابع، اعتبارسنجی با مدیریت خطا، کار با فایل و JSON، برنامه‌نویسی شیءگرا، ساختمان‌داده و در نهایت مینی‌پروژه‌های ترکیبی. هر تمرین یک صورت سؤال فارسی و یک پاسخ کامل به صورت کد پایتون دارد؛ سؤال‌ها فارسی‌اند اما کدها و پیام‌های ورودی/خروجی طبق عرف برنامه‌نویسی انگلیسی نوشته شده‌اند تا با محیط واقعی کار تمرین کنید.

پیشنهاد من: قبل از دیدن پاسخ هر تمرین، حداقل ۱۵ دقیقه خودتان فکر کنید و روی کاغذ یا ادیتور تست بنویسید؛ یادگیری واقعی همان‌جا اتفاق می‌افتد.

برای دیدن آموزش‌ها و پروژه‌های بیشتر، پیج اینستاگرام @mahshidsheidaeiوکانال تلگرام @mahshidsheidaei را دنبال کنید.

ارادتمند شما، مهشید شیدایی

بخش اول: شرط‌ها و منطق کاربردی (تمرین ۱ تا ۱۰)

در این بخش فقط با ورودی/خروجی و ساختارهای if-elif-else سر و کار داریم؛ هیچ حلقه یا تابعی لازم نیست. تمرکز روی ترجمه شرایط واقعی به منطق برنامه است.

تمرین ۱: ارزیابی قدرت رمز عبور

صورت سؤال: رمز عبوری دریافت کنید؛ اگر دست‌کم ۸ کاراکتر داشته باشد و هم حرف و هم عدد در آن باشد «قوی»، در غیر این صورت «ضعیف» اعلام کنید.

```
password = input("Enter password: ")
has_alpha = any(c.isalpha() for c in password)
has_digit = any(c.isdigit() for c in password)
if len(password) >= 8 and has_alpha and has_digit:
    print("Strong password")
else:
    print("Weak password")
```

تمرین ۲: تخفیف پلکانی خرید

صورت سؤال: مبلغ خرید را بگیرید؛ اگر بیش از ۱٬۰۰۰٬۰۰۰ تومان بود ۱۵٪، اگر بیش از ۵۰۰٬۰۰۰ بود ۷٪، در غیر این صورت بدون تخفیف، مبلغ قابل پرداخت را چاپ کنید.

```
amount = float(input("Enter purchase amount: "))
if amount > 1_000_000:
    discount = amount * 0.15
elif amount > 500_000:
    discount = amount * 0.07
else:
    discount = 0
print(f"Discount: {discount:.0f} | Payable: {amount - discount:.0f}")
```

تمرین ۳: اعتبارسنجی ساعت

صورت سؤال: ساعت و دقیقه را بگیرید و معتبر بودن زمان را بررسی کنید.

```
hour = int(input("Enter hour: "))
minute = int(input("Enter minute: "))
print("Valid time" if 0 <= hour <= 23 and 0 <= minute <= 59 else
"Invalid time")
```

تمرین ۴: هزینه ارسال سفارش

صورت سؤال: اگر مبلغ سفارش حداقل ۱۰۰ باشد ارسال رایگان است؛ در غیر این صورت برای سفارش فوری ۱۵ و سفارش عادی ۷ واحد هزینه بگیرید.

```
amount = float(input("Enter order amount: "))
urgent = input("Urgent delivery? (y/n): ").lower() == "y"
if amount >= 100:
    shipping = 0
else:
    shipping = 15 if urgent else 7
print(f"Shipping cost: {shipping}")
```

تمرین ۵: تعیین شیفت کاری

صورت سؤال: ساعت ورود کاربر را بگیرید؛ بین ۶ تا ۱۴ شیفت «صبح»، ۱۴ تا ۲۲ شیفت «عصر» و بقیه شیفت «شب» است.

```
hour = int(input("Enter entry hour (0-23): "))
if 6 <= hour < 14:
    print("Morning shift")
elif 14 <= hour < 22:
    print("Evening shift")
else:
    print("Night shift")
```

تمرین ۶: هزینه پارکینگ

صورت سؤال: مدت توقف خودرو را به ساعت بگیرید؛ تا ۲ ساعت هر ساعت ۵ واحد، پس از آن هر ساعت ۳ واحد است؛ حداکثر هزینه ۵۰ واحد می‌شود.

```
hours = float(input("Enter parked hours: "))
if hours <= 2:
    fee = hours * 5
else:
    fee = 10 + (hours - 2) * 3
print(f"Parking fee: {min(fee, 50):.0f}")
```

تمرین ۷: وضعیت موجودی محصول

صورت سؤال: تعداد موجودی کالا را بگیرید؛ صفر یعنی «ناموجود»، کمتر از ۱۰ یعنی «محدود» و بقیه «موجود» است.

```
stock = int(input("Enter stock count: "))
if stock == 0:
    print("Out of stock")
elif stock < 10:
    print("Limited stock")
else:
    print("In stock")
```

تمرین ۸: تشخیص سال کبیسه

صورت سؤال: یک سال میلادی دریافت کنید و با قاعده استاندارد (تقسیم‌پذیری بر ۴، استثنای صد و استثنای چهارسد) کبیسه بودن را مشخص کنید.

```
year = int(input("Enter a year: "))
is_leap = (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0)
print("Leap year" if is_leap else "Common year")
```

تمرین ۹: محاسبه اضافه‌کاری

صورت سؤال: ساعات کار و نرخ ساعتی را بگیرید؛ ساعات بیشتر از ۴۰ را با ضریب ۱٫۵ محاسبه کنید.

```
hours = float(input("Enter worked hours: "))
rate = float(input("Enter hourly rate: "))
if hours > 40:
    salary = 40 * rate + (hours - 40) * rate * 1.5
else:
    salary = hours * rate
print(f"Salary: {salary:.2f}")
```

تمرین ۱۰: تعیین بازه دمایی

صورت سؤال: دما را بگیرید؛ زیر صفر «یخ‌زده»، صفر تا ۳۰ «معمولی» و بالای ۳۰ «داغ» گزارش شود.

```
temp = float(input("Enter temperature: "))
if temp < 0:
    print("Frozen")
elif temp <= 30:
    print("Normal")
else:
    print("Hot")
```

بخش دوم: حلقه‌ها و شبیه‌سازی (تمرین ۱۱ تا ۲۰)

اینجا حلقه‌های for و while وارد می‌شوند: انباشت مقدار، شمارش معکوس، شبیه‌سازی فرایندها و کنترل ورودی کاربر تا رسیدن به شرط.

تمرین ۱۱: پس‌انداز ماهانه

صورت سؤال: مبلغ پس‌انداز ماهانه و هدف را بگیرید و محاسبه کنید بعد از چند ماه به هدف می‌رسید.

```
monthly = float(input("Monthly saving: "))
target = float(input("Target amount: "))
months = 0
balance = 0
while balance < target:
    balance += monthly
    months += 1
print(f"Target reached after {months} months")
```

تمرین ۱۲: شمارش معکوس پویا

صورت سؤال: عددی بگیرید و از آن تا ۱ با حلقه while شمارش معکوس چاپ کنید.

```
n = int(input("Enter start number: "))
while n >= 1:
    print(n)
    n -= 1
print("Liftoff!")
```

تمرین ۱۳: شبیه‌سازی رشد جمعیت

صورت سؤال: جمعیت اولیه، نرخ رشد سالانه درصدی و سقف را بگیرید؛ چند سال طول می‌کشد جمعیت از سقف عبور کند؟

```
population = float(input("Initial population: "))
growth = float(input("Annual growth %: ")) / 100
cap = float(input("Population cap: "))
years = 0
while population <= cap:
    population *= (1 + growth)
    years += 1
print(f"Cap exceeded after {years} years")
```

تمرین ۱۴: دستگاه فروش بلیت

صورت سؤال: با موجودی ۱۰ بلیت، در هر تکرار تعداد درخواست مشتری را بگیرید؛ اگر کافی بود بفروشید و موجودی را کم کنید، وگرنه پیام خطا بدهید. تا تمام شدن موجودی ادامه دهید.

```
stock = 10
while stock > 0:
    request = int(input(f"Tickets requested ({stock} left): "))
    if request <= stock:
        stock -= request
        print(f"Sold {request} tickets")
    else:
```

```
        print("Not enough tickets!")
print("Sold out")
```

تمرین ۱۵: محاسبه بهره مرکب

صورت سؤال: سرمایه اولیه، نرخ بهره سالانه و مدت را بگیرید و موجودی را در پایان هر سال با حلقه for چاپ کنید.

```
principal = float(input("Initial capital: "))
rate = float(input("Annual interest %: ")) / 100
years = int(input("Years: "))
balance = principal
for year in range(1, years + 1):
    balance *= (1 + rate)
    print(f"Year {year}: {balance:.2f}")
```

تمرین ۱۶: الگوی پله‌ای شماره‌دار

صورت سؤال: برای ورودی n، الگویی چاپ کنید که سطر i ام، شماره‌های ۱ تا i را نشان دهد.

```
n = int(input("Enter n: "))
for i in range(1, n + 1):
    print("".join(str(j) for j in range(1, i + 1)))
```

تمرین ۱۷: کنترل تلاش ورود

صورت سؤال: رمز درست سیستم «1234» است؛ کاربر حداکثر ۳ بار حق تلاش دارد. موفقیت یا قفل شدن حساب را گزارش کنید.

```
correct = "1234"
attempts = 0
while attempts < 3:
    if input("Enter PIN: ") == correct:
        print("Access granted")
```

```
        break
    attempts += 1
else:
    print("Account locked")
```

تمرین ۱۸: شبیه‌سازی مصرف باتری

صورت سؤال: باتری از ۱۰۰ درصد شروع می‌شود؛ هر ساعت ۱۵ درصد کم می‌شود. درصد باقی‌مانده در پایان هر ساعت را چاپ کنید تا زیر ۲۰ برسد.

```
battery = 100
hour = 0
while battery >= 20:
    hour += 1
    battery -= 15
    print(f"Hour {hour}: battery {max(battery, 0)}%")
print("Low battery warning!")
```

تمرین ۱۹: نخستین مضرب مشترک پس از حد

صورت سؤال: دو عدد a و b و یک حداقل مقدار را بگیرید؛ اولین مضرب مشترک آن‌ها که از حد بزرگ‌تر است را بیابید.

```
a = int(input("Enter a: "))
b = int(input("Enter b: "))
limit = int(input("Enter minimum value: "))
n = max(a, b)
while True:
    if n > limit and n % a == 0 and n % b == 0:
        print(f"First common multiple above {limit}: {n}")
        break
    n += 1
```

تمرین ۲۰: فروش روزانه تا رسیدن به هدف

صورت سؤال: فروش روزانه فروشگاه را از کاربر بگیرید و مجموع را نگه دارید؛ به محض عبور مجموع از هدف فروش، توقف و گزارش بدهید.

```
target = float(input("Enter sales target: "))
total = 0
days = 0
while total <= target:
    total += float(input(f"Day {days + 1} sales: "))
    days += 1
print(f"Target reached in {days} days, total = {total:.0f}")
```

بخش سوم: پردازش متن و رشته‌ها (تمرین ۲۱ تا ۳۰)

در این بخش با متدهای رشته‌ای، برش‌زنی (split، (slicing و join کار می‌کنیم؛ سناریوها از پاک‌سازی متن و اعتبارسنجی تا رمزنگاری ساده را پوشش می‌دهند.

تمرین ۲۱: پاک‌سازی فاصله‌های اضافی

صورت سؤال: متنی با فاصله‌های تکراری بگیرید و بین کلمات فقط یک فاصله باقی بگذارید.

```
text = input("Enter messy text: ")
cleaned = " ".join(text.split())
print("Cleaned:", cleaned)
```

تمرین ۲۲: شمارش هشتگ معتبر

صورت سؤال: یک متن شبکه اجتماعی بگیرید و تعداد ه (گ‌های معتبر (بدون فاصله و با دست‌کم یک حرف بعد از #) را بشمارید.

```
text = input("Enter post text: ")
tags = [w for w in text.split() if w.startswith("#") and len(w) > 1]
print(f"Valid hashtags: {len(tags)}")
```

تمرین ۲۳: پنهان‌سازی شماره کارت

صورت سؤال: شماره کارت ۱۶ رقمی بگیرید و فقط ۴ رقم آخر را نشان دهید و بقیه را با ستاره جایگزین کنید.

```
card = input("Enter 16-digit card number: ")
if len(card) == 16 and card.isdigit():
    print("Masked:", "*" * 12 + card[-4:])
else:
    print("Invalid card number")
```

تمرین ۲۴: ساخت نام کاربری

صورت سؤال: نام و نام خانوادگی بگیرید و نام کاربری به شکل حروف کوچک نام + نقطه + نام خانوادگی بسازید.

```
first = input("First name: ").strip().lower()
last = input("Last name: ").strip().lower()
print(f"Username: {first}.{last}")
```

تمرین ۲۵: استخراج دامنه ایمیل

صورت سؤال: یک ایمیل بگیرید و دامنه آن (بخش بعد از @) را چاپ کنید؛ اگر @ نداشت پیام خطا بدهید.

```
email = input("Enter email: ")
if "@" in email:
    print(f"Domain: {email.split('@')[-1]}")
else:
    print("Invalid email!")
```

تمرین ۲۶: کوتاه‌سازی عنوان

صورت سؤال: عنوانی بگیرید؛ اگر بیش از ۲۰ کاراکتر بود آن را به ۱۷ کاراکتر برش بزنید و سه نقطه اضافه کنید.

```
title = inputif len(title) > 20:
```

```
    title = title 20:
    title = title[:17] + "..."
print("Result:", title)
```

تمرین ۲۷: شمارش جمله‌ها

صورت سؤال: متنی بگیرید و تعداد جمله‌ها را بر اساس نقطه، علامت سؤال و تعجب بشمارید.

```
text = input("Enter text: ")
count = sum(text.count(p) for p in (".", "?", "!"))
print(f"Sentence count: {count}")
```

تمرین ۲۸: ساخت سرواژه

صورت سؤال: از هر کلمه یک نام کامل، حرف اول را گرفته و سرواژه با نقطه بین حروف بسازید.

```
name = input("Enter full name: ")
initials = ".".join(w[0].upper() for w in name.split())
print("Initials:", initials)
```

تمرین ۲۹: تشخیص حروف تکراری غیرمجاز

صورت سؤال: رشته‌ای بگیرید و بررسی کنید آیا کاراکتری بیش از یک بار تکرار شده یا نه (فقط حروف و اعداد؛ فاصله مهم نیست).

```
text = input("Enter a string: ").replace(" ", "").lower()
if len(set(text)) == len(text):
    print("No repeated characters")
else:
    print("Duplicates found!")
```

تمرین ۳۰: قالب‌بندی عنوان کتاب

صورت سؤال: عنوانی با حروف بی‌قاعده بگیرگیرید و هر کلمه را با حرف اول بزرگ و بقیه کوچک چاپ کنید (بدون تابع title).

```
text = input("Enter book title: ")
words = [w[0].upper() + w[1:].lower() if w else "" for w in text.split()]
print("Formatted:", " ".join(words))
```

بخش چهارم: فهرست، دیکشنری و مجموعه (تمرین ۳۱ تا ۴۰)

تمرین‌های این بخش سناریوهای فروشگاه، انبار و تیم‌کاری را با لیست، دیکشنری و مجموعه حل می‌کنند؛ مهارت اصلی انتخاب ساختار داده مناسب و پیمایش درست است.

تمرین ۳۱: ادغام موجودی انبار

صورت سؤال: دو دیکشنری موجودی کالا بگیرید و در یک دیکشنری ادغام کنید؛ برای کالای تکراری مجموع مقادیر ذخیره شود.

```
warehouse1 = {"laptop": 5, "mouse": 20, "keyboard": 12}
warehouse2 = {"laptop": 3, "monitor": 8, "keyboard": 4}
merged = dict(warehouse1)
for item, qty in warehouse2.items():
    merged[item] = merged.get(item, 0) + qty
print("Merged inventory:", merged)
```

تمرین ۳۲: گروه‌بندی نام‌ها بر اساس حرف اول

صورت سؤال: لیستی از نام‌ها را در یک دیکشنری بر اساس حرف اولشان گروه‌بندی کنید.

```
names = ["Ali", "Akbar", "Sara", "Sina", "Reza", "Mina", "Mahdi"]
grouped = {}
for name in names:
    grouped.setdefault(name[0].lower(), []).append(name)
print("Grouped:", grouped)
```

تمرین ۳۳: کالاهای کم‌موجود

صورت سؤال: از دیکشنری موجودی، همه کالاهایی که موجودی‌شان کمتر از ۵ است را در یک لیست جدا کنید.

```
inventory = {"pen": 3, "notebook": 15, "eraser": 2, "marker": 9, "clip": 4}
low = [item for item, qty in inventory.items() if qty < 5]
print("Low stock items:", low)
```

تمرین ۳۴: اشتراک مهارت‌های تیم

صورت سؤال: مجموعه مهارت‌های دو تیم را بگیرید و مهارت‌هایی که هر دو تیم دارند را چاپ کنید.

```
team_a = {"python", "sql", "git", "docker"}
team_b = {"python", "linux", "git", "react"}
print("Shared skills:", team_a & team_b)
```

تمرین ۳۵: رتبه‌بندی فروشندگان

صورت سؤال: دیکشنری فروش فروشندگان را از بیشترین به کمترین مرتب و همراه رتبه چاپ کنید.

```
sales = {"Ali": 520, "Sara": 810, "Reza": 430, "Mina": 690}
ranked = sorted(sales.items(), key=lambda x: x[1], reverse=True)
for rank, (name, amount) in enumerate(ranked, start=1):
    print(f"Rank {rank}: {name} - {amount}")
```

تمرین ۳۶: تقسیم به بسته‌های ثابت

صورت سؤال: لیستی از اقلام را به بسته‌های n تایی تقسیم کنید (آخرین بسته می‌تواند ناقص باشد).

```
items = list(range(1, 11))
size = 4
packs = [items[i:i + size] for i in range(0, len(items), size)]
```

```
print("Packs:", packs)
```

تمرین ۳۷: چرخش فهرست

صورت سؤال: یک لیست را به اندازه k گام به سمت چپ بچرخانید (عناصر اول به انتها می‌روند).

```
lst = [1, 2, 3, 4, 5, 6, 7]
k = int(input("Rotate by: ")) % len(lst)
print("Rotated:", lst[k:] + lst[:k])
```

تمرین ۳۸: فهرست انتظار بدون تکرار

صورت سؤال: نام‌های ثبت‌نامی را یکی‌یکی بگیرید؛ اگر قبلاً ثبت نشده بود به فهرست انتظار اضافه شود، وگرنه پیام تکراری بودن چاپ شود.

```
waiting = []
for _ in range(5):
    name = input("Applicant name: ")
    if name in waiting:
        print(f"{name} is already registered!")
    else:
        waiting.append(name)
        print(f"{name} added")
print("Final waiting list:", waiting)
```

تمرین ۳۹: ماتریس فروش هفتگی

صورت سؤال: فروش هفتگی ۳ فروشگاه در ۴ روز را در یک لیست دوبعدی نگه دارید و مجموع فروش هر فروشگاه و روز پرفروش هرکدام را چاپ کنید.

```
sales = [
    [100, 220, 180, 90],
    [80, 150,n    [200, 160, 110, 240],, 110, 240],
]
```

```
for i, row in enumerate(sales, start=1):
    print(f"Store {i}: total={sum(row)}, best_day={row.index(max(row))
+ 1}")
```

تمرین ۴۰: اختلاف دو سبد خرید

صورت سؤال: اقصورت سؤال:** اقلام دو سبد خرید را بگیرید و اق دو سبد هست (نه هر دو) را چاپ کنید.

```
cart1 = {"milk", "bread", "cheese", "tea"}
cart2 = {"bread", "honey", "tea", "butter"}
print("Only in one cart:", cart1 ^ cart2)
```

بخش پنجم: توابع و الگوریتم‌ها (تمرین ۴۱ تا ۵۰)

از این بخش به بعد منطق برنامه‌ها را در تابع‌های مستقل می‌نویسیم؛ تمرکز روی پارامتر و مقدار بازگشتی، الگوریتم‌های کلاسیک و کاربرد ماژول‌های آماده است.

تمرین ۴۱: قیمت پس از مالیات

صورت سؤال: تابعی بنویسید که قیمت پایه و درصد مالیات را بگیرد و قیمت نهایی را برگرداند؛ سپس نتیجه را برای چند کالا چاپ کنید.

```
def price_with_tax(price, tax_percent):
    return price * (1 + tax_percent / 100)

for base in (100, 250, 999):
    print(f"Base: {base} -> Final: {price_with_tax(base, 9):.2f}")
```

تمرین ۴۲: تبدیل بازه امتیاز

صورت سؤال: تابعی بنویسید که عدد ۰ تا ۱۰۰ را به حرف (A از ۹۰، B از ۸۰، C از ۷۰، D از ۶۰ و بقیه F) تبدیل کند.

```
def to_letter_grade(score):
    if score >= 90: return "A"
```

```
    elif score >= 80: return "B"
    elif score >= 70: return "C"
    elif score >= 60: return "D"
    return "F"

for s in (95, 82, 71, 63, 40):
    print(s, "->", to_letter_grade(s))
```

تمرین ۴۳: نزدیک‌ترین مقدار

صورت سؤال: تابعی بنویسید که در یک لیست عددی، عددی را بیابد که کمترین فاصله را با مقدار هدف دارد.

```
def closest(numbers, target):
    return min(numbers, key=lambda x: abs(x - target))

nums = [4, 18, 22, 7, 31]
print(f"Closest to 20: {closest(nums, 20)}")
```

تمرین ۴۴: فشرده‌سازی بازه‌های عددی

صورت سؤال: تابعی بنویسید که لیست مرتب‌شده‌ای از اعداد را به بازه‌های پیوسته تبدیل کند؛ مثلا [1,2,3,7,8] به «1-3,7-8».

```
def compress_ranges(nums):
    if not nums:
        return ""
    start = prev = nums[0]
    ranges = []
    for n in nums[1:]:
        if n != prev + 1:
            ranges.append(f"{start}-{prev}" if start != prev else str(start))
            start = n
        prev = n
    ranges.append(f"{start}-{prev}" if start != prev else str(start))
```

```
    return ",".join(ranges)

print(compress_ranges([1, 2, 3, 7, 8, 15]))
```

تمرین ۴۵: اقساط مساوی وام

صورت سؤال: تابعی بنویسید که مبلغ وام، نرخ سود سالانه و تعداد اقساط را بگیرد و مبلغ هر قسط (روش ساده: اصل تقسیم بر اقساط به‌علاوه سود باقیمانده به‌صورت میانگین) را برگرداند.

```
def installment(loan, annual_rate, months):
    total = loan * (1 + annual_rate / 100 * (months / 12))
    return total / months

print(f"Monthly installment: {installment(120_000_000, 18, 12):.0f}
toman")
```

تمرین ۴۶: ادغام بازه‌های هم‌پوشان

صورت سؤال: لیستی از بازه‌های [شروع، پایان] بگیرید و بازه‌های هم‌پوشان را ادغام کنید.

```
def merge_intervals(intervals):
    intervals = sorted(intervals)
    merged = [intervals[0]]
    for start, end in intervals[1:]:
        if start <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])
    return merged

print(merge_intervals([[1, 4], [3, 6], [8, 10]]))
```

تمرین ۴۷: تعادل پرانتزها

صورت سؤال: تابعی بنویسید که بررسی کند پرانتزهای یک عبارت متعادل هستند یا نه.

```
def is_balanced(expr):
    depth = 0
    for ch in expr:
        if ch == "(":
            depth += 1
        elif ch == ")":
            depth -= 1
            if depth < 0:
                return False
    return depth == 0

print(is_balanced("(a + (b * c))"))
print(is_balanced(")a("))
```

تمرین ۴۸: زمان انتظار میانگین

صورت سؤال: تابعی بنویسید که لیست زمان‌های رسیدن مشتریان و زمان خدمت هرکدام را بگیرد و میانگین زمان انتظار را بر اساس الگوریتم اول بدم‌اول‌خدمت محاسبه کند.

```
def avg_waiting_time(service_times):
    waiting, elapsed = [], 0
    for t in service_times:
        waiting.append(elapsed)
        elapsed += t
    return sum(waiting) / len(waiting)

print(f"Avg wait: {avg_waiting_time([5, 3, 8]):.1f} minutes")
```

تمرین ۴۹: طولانی‌ترین روند افزایشی

صورت سؤال: تابعی بنویسید که طول طولانی‌ترین دنباله پیوسته اعداد صعودی در یک لیست را برگرداند.

```
def longest_run(nums):
    best = current = 1
    for i in range(1, len(nums)):
        current = current + 1 if nums[i] > nums[i-1] else 1
        best = max(best, current)
    return best

print(longest_run([1, 3, 5, 4, 7, 8, 9, 2]))
```

تمرین ۵۰: تخصیص بودجه تناسبی

صورت سؤال: تابعی بنویسید که مبلغ کل بودجه را متناسب با امتیاز هر واحد تقسیم کند و سهم هر واحد را گردشده چاپ کند.

```
def allocate(budget, weights):
    total = sum(weights)
    return {unit: round(budget * w / total) for unit, w in
weights.items()}

shares = allocate(10_000_000, {"IT": 5, "HR": 2, "Sales": 3})
print(shares)
```

بخش ششم: اعتبارسنجی و مدیریت خطا (تمرین ۵۱ تا ۶۰)

تمرین‌های این بخش ورودی نامعتبر را به صورت کنترل‌شده مدیریت می‌کنند؛ ترکیب try-except با حلقه‌های تکرار ورودی و اعتبارسنجی با عبارت باقاعده (re) مهارت اصلی این بخش است.

صورت سؤال: تا عدد صحیح در بازه
صورت سؤال: تا زمانی که کاربر عدد صحیحی بین ۱ تا ۱۰۰ وارد نکرده، ورودی را با پیام مناسب تکرار کنید.

```
while True:
    try:
        value = int(input("Enter a number (1-100): "))
```

```
        if 1 <= value <= 100:
            print(f"Accepted: {value}")
            break
        print("Out of range!")
    except ValueError:
        print("Please enter a valid integer!")
```

تمرین ۵۲: اعتبارسنجی شماره تلفن

صورت سؤال: با ماژول re بررسی کنید شماره موبایل در قالب ۱۱) 09xxxxxxxxx رقم و شروع با 09) است یا نه.

```
import re

phone = input("Enter mobile number: ")
if re.fullmatch(r"09\d{9}", phone):
    print("Valid mobile number")
else:
    print("Invalid mobile number")
```

تمرین ۵۳: اعتبارسنجی تاریخ میلادی

صورت سؤال: سال، ماه و روز را بگیرید و با try-except و ماژول datetime معتبر بودن تاریخ را بررسی کنید.

```
from datetime import date

try:
    y = int(input("Year: ")):n    m = int(input("Month: "))
    d = int(input("Day: "))
    date(y, m, d)
    print("Valid date")
except ValueError as e:
    print(f"Invalid date: {e}")
```

تمرین ۵۴: تقسیم امن متنی

صورت سؤال: دو عدد متنی بگیرید و اولی را بر دومی تقسیم کنید؛ صفر بودن مقسوم‌علیه یا عددی نبودن ورودی را با پیام جدا مدیریت کنید.

```
try:
    a = float(input("Numerator: "))
    b = float(input("Denominator: "))
    print(f"Result: {a / b:.2f}")
except ZeroDivisionError:
    print("Cannot divide by zero!")
except ValueError:
    print("Inputs must be numbers!")
```

تمرین ۵۵: اعتبارسنجی کد محصول

صورت سؤال: کد محصول باید از سه حرف بزرگ، خط تیره و چهار رقم تشکیل شود؛ با re اعتبارسنجی کنید.

```
import re

code = input("Enter product code: ")
if re.fullmatch(r"[A-Z]{3}-\d{4}", code):
    print("Valid product code")
else:
    print("Invalid product code (expected ABC-1234)")
```

تمرین ۵۶: دریافت انتخاب از منو

صورت سؤال: منویی سه‌گزینه‌ای نمایش دهید و تا انتخاب عددی معتبر (۱ تا ۳) دریافت نشده، تکرار کنید.

```
menu = {"1": "Deposit", "2": "Withdraw", "3": "Exit"}
while True:
    print("Menu:", " | ".join(f"{k}. {v}" for k, v in menu.items()))
    choice = input("Select: ")
```

```
    if choice in menu:
        print("You chose:", menu[choice])
        break
    print("Invalid choice!")
```

تمرین ۵۷: اعتبارسنجی IPv4

صورت سؤال: رشته‌ای بگیرید و با تقسیم بر نقطه بررسی کنید آیا یک IPv4 معتبر (چهار عدد ۰ تا ۲۵۵) است یا نه.

```
def is_valid_ipv4(ip):
    parts = ip.split(".")
    if len(parts) != 4:
        return False
    return all(p.isdigit() and 0 <= int(p) <= 255 for p in parts)

print(is_valid_ipv4("192.168.1.1"))
print(is_valid_ipv4("256.10.1.3"))
```

تمرین ۵۸: تبدیل مبلغ با پیام خطای دقیق

صورت سؤال: متنی بگیرید و آن را به عدد تبدیل کنید؛ در صورت خطا نوع دقیق مشکل (خالی بودن، حروف غیرعددی) را چاپ کنید.

```
raw = input("Enter amount: ").strip()
if not raw:
    print("Error: input is empty")
else:
    try:
        print(f"Amount: {float(raw):,.0f}")
    except ValueError:
        print(f"Error: '{raw}' is not a number")
```

تمرین ۵۹: بررسی مسیر فایل ورودی

صورت سؤال: مسیری از کاربر بگیرید و با pathlib بررسی کنید وجود دارد، فایل است یا پوشه؛ خطاهای احتمالی را مدیریت کنید.

```
from pathlib import Path

path = Path(input("Enter a path: "))
if path.exists() and path.is_file():
    print(f"File found ({path.stat().st_size} bytes)")
elif path.is_dir():
    print("It is a directory")
else:
    print("Path does not exist")
```

تمرین ۶۰: اعتبارسنجی رکورد کاربر

صورت سؤال: رکوردی شامل نام، سن و ایمیل را بگیرید؛ نام خالی، سن غیرعددی یا ایمیل بدون @ را با چند except جداگانه مدیریت کنید.

```
def validate_user(name, age, email):
    if not name.strip():
        raise ValueError("Name is empty")
    if not age.isdigit():
        raise TypeError("Age must be a number")
    if "@" not in email:
        raise ValueError("Email must contain @")
    return True

try:
    validate_user(input("Name: "), input("Age: "), input("Email: "))
    print("User record is valid")
except (ValueError, TypeError) as err:
    print("Invalid record:", err)
```

بخش هفتم: فایل، CSV و JSON (تمرین ۶۱ تا ۷۰)

در این بخش با خواندن و نوشتن فایل‌های متنی و ساختارهای CSV و JSON کار می‌کنیم؛ الگوی رایج، بلوک with برای باز و بسته شدن خودکار فایل و مدیریت FileNotFoundError است.

تمرین ۶۱: شمارش خطوط غیرخالی

صورت سؤال: یک فایل متنی را بخوانید و تعداد خطوط غیرخالی و پرتکرارترین کلمه آن را چاپ کنید.

```
from pathlib import Path

content = Path("notes.txt").read_text(encoding="utf-8")
non_empty = [line for line in content.splitlines() if line.strip()]
print(f"Non-empty lines: {len(non_empty)}")
```

تمرین ۶۲: افزودن گزارش رویداد

صورت سؤال: تابعی بنویسید که هر بار یک پیام با تاریخ و ساعت جاری به انتهای فایل log.txt اضافه کند (حالت append).

```
from datetime import datetime

def log_event(message):
    with open("log.txt", "a", encoding="utf-8") as f:
        f.write(f"{datetime.now():%Y-%m-%d %H:%M} | {message}\n")

log_event("Application started")
log_event("User logged in")
```

تمرین ۶۳: مجموع فروش CSV

صورت سؤال: فایل CSV با ستون‌های product و amount را بخوانید و مجموع فروش هر محصول را چاپ کنید.

```
import csv
```

```
sales = {}
with open("sales.csv", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        sales[row["product"]] = sales.get(row["product"], 0) +
float(row["amount"])
for product, total in sales.items():
    print(f"{product}: {total:.0f}")
```

تمرین ۶۴: فیلتر CSV و ساخت فایل جدید

صورت سؤال: از فایل دانشجویان، رکوردهایی که معدل بالای ۱۷ دارند را در فایل جدید top_students.csv بنویسید.

```
import csv

with open("students.csv", encoding="utf-8") as src, \
     open("top_students.csv", "w", newline="", encoding="utf-8") as
dst:
    reader = csv.DictReader(src)
    writer = csv.DictWriter(dst, fieldnames=reader.fieldnames)
    writer.writeheader()
    for row in reader:
        if float(row["gpa"]) > 17:
            writer.writerow(row)
print("top_students.csv created")
```

تمرین ۶۵: به‌روزرسانی تنظیمات JSON

صورت سؤال: فایل تنظیمات JSON را بخوانید، مقدار یک کلید را تغییر دهید و دوباره ذخیره کنید؛ فایل نبودن را مدیریت کنید.

```
import json
from pathlib import Path

path = Path("config.json")
```

```
settings = json.loads(path.read_text(encoding="utf-8")) if
path.exists() else {}
settings["theme"] = "dark"
settings["language"] = "fa"
path.write_text(json.dumps(settings, indent=2, ensure_ascii=False),
encoding="utf-8")
print("Settings saved:", settings)
```

تمرین ۶۶: پشتیبان‌گیری از فایل

صورت سؤال: تابعی بنویسید که از یک فایل، نسخه پشتیبان با پسوند .bak بسازد و پیام موفقیت یا خطا بدهد.

```
import shutil
from pathlib import Path

def backup(filename):
    src = Path(filename)
    if not src.exists():
        print(f"Error: {filename} not found")
        return
    shutil.copy(src, src.with_suffix(src.suffix + ".bak"))
    print(f"Backup created: {src}.bak")

backup("data.txt")
```

تمرین ۶۷: بزرگ‌ترین فایل پوشه

صورت سؤال: در یک پوشه، فایلی با بیشترین حجم را پیدا و نام و اندازه آن را چاپ کنید.

```
from pathlib import Path

folder = Path(".")
files = [f for f in folder.iterdir() if f.is_file()]
if files:
```

```
    biggest = max(files, key=lambda f: f.stat().st_size)
    print(f"Largest file: {biggest.name} ({biggest.stat().st_size}
bytes)")
else:
    print("No files found")
```

تمرین ۶۸: تبدیل CSV به JSON

صورت سؤال: داده‌های یک فایل CSV را خوانده و به فایل JSON تبدیل و ذخیره کنید.

```
import csv, json

with open("products.csv", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))
with open("products.json", "w", encoding="utf-8") as f:
    json.dump(rows, f, indent=2, ensure_ascii=False)
print(f"Converted {len(rows)} records to JSON")
```

تمرین ۶۹: استخراج خطوط ERROR از گزارش

صورت سؤال: از فایل گزارش سرور، فقط خطوطی که شامل ERROR هستند را در فایل جداگانه بنویسید و تعدادشان را چاپ کنید.

```
error_lines = []
with open("server.log", encoding="utf-8") as f:
    for line in f:
        if "ERROR" in line:
            error_lines.append(line)
with open("errors.log", "w", encoding="utf-8") as f:
    f.writelines(error_lines)
print(f"{len(error_lines)} error lines extracted")
```

تمرین ۷۰: فراوانی پسوند فایل‌ها

صورت سؤال: در یک پوشه، تعداد فایل‌های هر پسوند را با Counter بشمارید و مرتب چاپ کنید.

```
from pathlib import Path
from collections import Counter

extensions = Counter(f.suffix.lower() for f in Path(".").iterdir() if
f.is_file())
for ext, count in extensions.most_common():
    print(f"{ext or 'no ext'}: {count}")
```

بخش هشتم: شیءگرایی کاربردی (تمرین ۷۱ تا ۸۰)

اینجا مفاهیم شیءگرا را در سناریوهای واقعی تمرین می‌کنیم: سازنده، متدها، ویژگی محاسباتی با @property و چندریختی. تمرکز بر مدل‌سازی درست موجودیت‌های دنیای واقعی است.

تمرین ۷۱: کیف پول دیجیتال

صورت سؤال: کلاسی بنویسید که موجودی را نگه دارد و متدهای deposit و withdraw داشته باشد؛ برداشت بیش از موجودی مجاز نیست.

```
class Wallet:
    def __init__(self, balance=0):
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient funds!")
        else:
            self.balance -= amount

wallet = Wallet(100)
```

```
wallet.deposit(50)
wallet.withdraw(200)
print(f"Balance: {wallet.balance}")
```

تمرین ۷۲: محصول با تخفیف

صورت سؤال: کلاس Product با نام و قیمت بسازید؛ متد apply_discount درصد تخفیف بگیرد و قیمت جدید را اعمال کند.

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

    def apply_discount(self, percent):
        self.price *= (1 - percent / 100)
        print(f"{self.name} new price: {self.price:.0f}")

p = Product("Laptop", 45_000_000)
p.apply_discount(10)
```

تمرین ۷۳: حساب بانکی با تاریخچه

صورت سؤال: کلاس حساب بانکی بسازید که هر تراکنش را با نوع و مبلغ در یک لیست تاریخچه ذخیره کند و متدی برای نمایش آن داشته باشد.

```
class BankAccount:
    def __init__(self, owner):
        self.owner = owner
        self.balance = 0
        self.history = []

    def transact(self, kind, amount):
        self.balance += amount if kind == "deposit" else -amount
        self.history.append((kind, amount))
```

```
    def show_history(self):
        for kind, amount in self.history:
            print(f"{kind}: {amount}")

acc = BankAccount("Ali")
acc.transact("deposit", 500)
acc.transact("withdraw", 120)
acc.show_history()
```

تمرین ۷۴: کتاب امانتی

صورت سؤال: کلاس کتاب با وضعیت امانت بسازید؛ متدهای borrow و return_book وضعیت را تغییر دهند و امانت دوباره کتاب موجود را خطا بدهند.

```
class Book:
    def __init__(self, title):
        self.title = title
        self.is_borrowed = False

    def borrow(self):
        if self.is_borrowed:
            print(f"'{self.title}' is already borrowed!")
        else:
            self.is_borrowed = True
            print(f"'{self.title}' borrowed")

    def return_book(self):
        self.is_borrowed = False

b = Book("Python Crash Course")
b.borrow()
b.borrow()
b.return_book()
```

تمرین ۷۵: سبد خرید

صورت سؤال: کلاس سبد خرید بسازید که اقلام با نام، قیمت و تعداد را ذخیره و جمع کل را محاسبه کند.

```
class ShoppingCart:
    def __init__(self):
        self.items = []

    def add(self, name, price, quantity=1):
        self.items.append({"name": name, "price": price, "quantity":
quantity})

    def total(self):
        return sum(item["price"] * item["quantity"] for item in self.items)

cart = ShoppingCart()
cart.add("Book", 250_000, 2)
cart.add("Pen", 30_000, 3)
print(f"Total: {cart.total():,} toman")
```

تمرین ۷۶: کارمند و حقوق خالص

صورت سؤال: کلاس کارمند با حقوق پایه و بیمه بسازید؛ ویژگی net_salary با @property حقوق پس از کسر ۷ درصد بیمه و ۱۰ درصد مالیات را برگرداند.

```
class Employee:
    def __init__(self, name, base_salary):
        self.name = name
        self.base_salary = base_salary

    @property
    def net_salary(self):
        return self.base_salary * 0.83  # 7% insurance + 10% tax

emp = Employee("Sara", 20_000_000)
```

```
print(f"{emp.name} net salary: {emp.net_salary:,.0f}")
```

تمرین ۷۷: رزرو صندلی

صورت سؤال: کلاس تالار نمایش با ظرفیت بسازید؛ متد reserve صندلی رزرو کند و با پر شدن ظرفیت پیام تیمت بدهد.

```
class Theater:
    def __init__(self, capacity):
        self.capacity = capacity
        self.reserved = 0

    def reserve(self, seats=1):
        if self.reserved + seats > self.capacity:
            print("Theater is full!")
        else:
            self.reserved += seats
            print(f"Reserved {seats} seat(s)".format(seats=seats))

t = Theater(3)
for _ in range(4):
    t.reserve()
```

تمرین ۷۸: دماسنج با ویژگی محاسباتی

صورت سؤال: کلاس دماسنج دمای سانتی‌گراد را ذخیره کند (با اعتبارسنجی نبودن دمای زیر ۲۷۳-) و دمای فارنهایت را با @property محاسبه کند.

```
class Thermometer:
    def __init__(self, celsius=0):
        self.celsius = celsius

    @property
    def celsius(self):
        return self._celsius
```

```
    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("Below absolute zero!")
        self._celsius = value

    @property
    def fahrenheit(self):
        return self.celsius * 1.8 + 32

t = Thermometer(25)
print(f"Fahrenheit: {t.fahrenheit:.1f}")
```

تمرین ۷۹: اعلان‌های چندنوعی

صورت سؤال: کلاس‌های EmailNotifier و SMSNotifier با متد مشترک send بسازید و با یک حلقه روی لیست، پیام را به همه اعلان کنید (چندریختی).

```
class EmailNotifier:
    def send(self, msg):
        print(f"Email: {msg}")

class SMSNotifier:
    def send(self, msg):
        print(f"SMS: {msg}")

notifiers = [EmailNotifier(), SMSNotifier()]
for n in notifiers:
    n.send("Your order has been shipped")
```

تمرین ۸۰: دانشجو با میانگین وزنی

صورت سؤال: کلاس دانشجو بسازید که درس‌ها با تعداد واحد و نمره ذخیره کند و معدل وزنی را محاسبه کند.

```
class Student:
    def __init__(self, name):
        self.name = name
        self.courses = []

    def add_course(self, credits, grade):
        self.courses.append((credits, grade))

    @property
    def weighted_gpa(self):
        total_credits = sum(c for c, _ in self.courses)
        return sum(c * g for c, g in self.courses) / total_credits

s = Student("Mina")
s.add_course(3, 18)
s.add_course(2, 19)
s.add_course(4, 16)
print(f"{s.name} GPA: {s.weighted_gpa:.2f}")
```

بخش نهم: ساختمان داده و حل مسئله (تمرین ۸۱ تا ۹۰)

در این بخش ساختمان‌های داده کلاسیک و الگوریتم‌های جست‌وجو، مرتب‌سازی و برنامه‌نویسی پویا را با ابزار استاندارد پایتون (collections و heapq) پیاده‌سازی می‌کنیم.

تمرین ۸۱: صف پشتیبانی

صورت سؤال: با deque یک صف پشتیبانی بسازید؛ مشتریان اضافه شوند و به ترتیب ورود خدمت‌رسانی و خارج شوند.

```
from collections import deque

queue = deque()
queue.append("Ali")
queue.append("Sara")
queue.append("Reza")
```

```
while queue:
    print(f"Now serving: {queue.popleft()}")
print("Queue is empty")
```

تمرین ۸۲: تاریخچه مرورگر با پشته

صورت سؤال: با لیست، پشته تاریخچه مرورگر را پیاده‌سازی کنید: visit صفحه اضافه کند و back آخرین صفحه را بردارد و برگرداند.

```
history = []

def visit(page):
    history.append(pagen
def visit(page):
    history.append(page)
    print(f"Visited:n    if history:
        print(f"Back to: {history.pop()}")
    else:
        print("No history!")

visit("google.com")
visit("github.com")
back()
print("Current:", history[-1])
```

تمرین ۸۳: جست‌وجوی دودویی

صورت سؤال: تابعی بازگشتی بنویسید که در یک لیست مرتب با الگوریتم جست‌وجوی دودویی مقدار هدف را بیابد و ایندکس آن را برگرداند.

```
def binary_search(arr, target, low=0, high=None):
    if high is None:
        high = len(arr) - 1
    if low > high:
        return -1
```

```
    mid = (low + high) // 2
    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search(arr, target, mid + 1, high)
    else:
        return binary_search(arr, target, low, mid - 1)

nums = [2, 5, 8, 12, 16, 23, 38]
print(f"Index of 16: {binary_search(nums, 16)}")
```

تمرین ۸۴: مرتب‌سازی انتخابی

صورت سؤال: الگوریتم مرتب‌سازی انتخابی را دستی پیاده‌سازی کنید و وضعیت لیست را در هر پاس نشان دهید.

```
def selection_sort(arr):
    for i in range(len(arr)):
        min_index = i
        for j in range(i + 1, len(arr)):
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i]
        print(f"Pass {i + 1}: {arr}")
    return arr

print(selection_sort([64, 25, 12, 22, 11]))
```

تمرین ۸۵: حذف تکراری‌ها با مجموعه

صورت سؤال: از یک لیست بزرگ از اعداد، عناصر تکراری را حذف کرده و لیست مرتب بدون تکرار برگردانید.

```
def deduplicate_sorted(nums):
    return sorted(set(nums))
```

```
raw = [3, 7, 1, 3, 9, 7, 2, 1, 5]
print("Unique sorted:", deduplicate_sorted(raw))
```

تمرین ۸۶: شمارش فراوانی با Counter

صورت سؤال: با کلاس Counter از ماژول collections، پرفروش‌ترین کالاهای یک فروشگاه را پیدا و چاپ کنید.

```
from collections import Counter

orders = ["tea", "coffee", "tea", "cake", "coffee", "tea", "juice"]
counter = Counter(orders)
print("Top sellers:", counter.most_common(2))
```

تمرین ۸۷: صف اولویت با heapq

صورت سؤال: با heapq صف اولویت‌دار بسازید که بیمارانی با کد فوریت (عدد کوچک‌تر = اولویت بیشتر) را به ترتیب خدمت کند.

```
import heapq

patients = [(3, "Broken arm"), (1, "Heart attack"), (2, "High fever")]
heapq.heapify(patients)
while patients:
    urgency, name = heapq.heappop(patients)
    print(f"Treating (P{urgency}): {name}")
```

تمرین ۸۸: معتبربودن پرانتزهای چندنوعی

صورت سؤال: تابعی بنویسید که تعادل پرانتزهای (), [] و {} در یک عبارت را با پشته بررسی کند.

```
def is_valid_brackets(expr):
    pairs = {")": "(", "]": "[", "}": "{"}
```

```
    stack = []
    for ch in expr:
        if ch in "([{":
            stack.append(ch)
        elif ch in ") "([{":
            stack.append(ch)
        elif ch in ")]:
                return False
    return not stack

print(is_valid_brackets("{[()]}"))
print(is_valid_brackets("{[(])}"))
```

تمرین ۸۹: بزرگ‌ترین زیرآرایه (کادِن)

صورت سؤال: الگوریتم کادِن را پیاده‌سازی کنید که مجموع بزرگ‌ترین زیرآرایه پیوسته را در O(n) بیابد.

```
def max_subarray(nums):
    best = current = nums[0]
    for n in nums[1:]:
        current = max(n, current + n)
        best = max(best, current)
    return best

print(max_subarray([-2, 1, -3, 4, -1, 2, 1, -5, 4]))
```

تمرین ۹۰: برنامه‌نویسی پویا: جمع غیرمجاور

صورت سؤال: تابع max_non_adjacent_sum بنویسید که بزرگ‌ترین جمع عناصر غیرمجاور یک لیست را با برنامه‌نویسی پویا محاسبه کند.

```
def max_non_adjacent_sum(nums):
    include, exclude = 0, 0
    for n in nums:
```

```
        include, exclude = exclude + n, max(include, exclude)
    return max(include, exclude)

print(max_non_adjacent_sum([3, 2, 7, 10]))
print(max_non_adjacent_sum([5, 5, 10, 100, 10, 5]))
```

بخش دهم: مینی‌پروژه‌های کاربردی (تمرین ۹۱ تا ۱۰۰)

ده تمرین پایانی، همه مفاهیم کتاب را در قالب پروژه‌های کوچک اما واقعی ترکیب می‌کنند: ورودی، اعتبارسنجی، ساختار داده، تابع و خروجی گزارش‌دهی.

تمرین ۹۱: مدیریت هزینه‌های روزانه

صورت سؤال: برنامه‌ای بنویسید که هزینه‌های روزانه کاربر را با توضیح بگیرد، در فهرستی ذخیره کند و در پایان مجموع، بیشترین هزینه و میانگین روزانه را گزارش کند.

```
expenses = []
while True:
    entry = input("Expense and description (or 'done'): ")
    if entry.lower() == "done":
        break
    parts = entry.split(" ", 1)
    expenses.append((float(parts[0]), parts[1]))

if expenses:
    total = sum(amount for amount, _ in expenses)
    biggest = max(expenses)
    print(f"Total: {total:,.0f} | Average: {total / len(expenses):,.0f}")
    print(f"Biggest expense: {biggest[1]} ({biggest[0]:,.0f})")
```

تمرین ۹۲: بازی حدس عدد

صورت سؤال: بازی حدس عدد بسازید: سیستم عددی ۱ تا ۱۰۰ بازی حدس عدد بسازید: سیستم عددی ۱ تا ۱۰۰الاتر» یا «پایین‌تر» بگوید؛ در پایان تعداد تلاش‌ها گزارش شود.

```
import random

secret = random.randint(1, 100)
attempts = 0
while True:
    guess = int(input("Your guess: "))
    attempts += 1
    if guess == secret:
        print(f"Correct! Attempts: {attempts}")
        break
    print("Higher!" if guess < secret else "Lower!")
```

تمرین ۹۳: سیستم رأی‌گیری با Counter

صورت سؤال: برنامه‌ای بنویسید که نام گزینه‌ها را از کاربران بگیرد و در پایان با Counter رأی‌ها را بشمارد و برنده را اعلام کند.

```
from collections import Counter

votes = Counter()
candidates = ["Ali", "Sara", "Reza"]
print("Candidates:", candidates)
while True:
    vote = input("Vote (or 'end'): ").capitalize()
    if vote == "End":
        break
    if vote in candidates:
        votes[vote] += 1
    else:
        print("Invalid candidate        votes[vote] += 1
    else:
        print("Invalid candidate: {winner[0][0]} with {winner[0][1]}
votes")
```

تمرین ۹۴: دفترچه مخاطبان

صورت سؤال: برنامه منوداری بسازید که با دیکشنری، مخاطب اضافه کند، جست‌وجو کند، حذف کند و لیست کامل را نمایش دهد؛ تا انتخاب گزینه خروج ادامه دهد.

```
contacts = {}
while True:
    print("1.Add 2.Search 3.Delete 4.Show 5.Exit")
    choice = input("Select: ")
    if choice == "1":
        name = input("Name: ")
        contacts[name] = input("Phone: ")
    elif choice == "2":
        name = input("Name: ")
 name = input("Name: ")
"))
    elif choice == "3":
        name = input("Name: ")
        print("Deleted" if contacts.pop(name, None) else "Not found")
    elif choice == "4":
        for name, phone in contacts.items():
            print(f"{name}: {phone}")
    elif choice == "5":
        break
```

تمرین ۹۵: تبدیل ارز با نرخ ثابت

صورت سؤال: برنامه‌ای بنویسید که با نرخ‌های ثابت، مبلغ تومانی را به دلار، یورو و درهم تبدیل کند و جدول خروجی چاپ کند.

```
rates = {"USD": 61000, "EUR": 66000, "AED": 16600}
amount = float(input("Amount in toman: "))
for currency, rate in rates.items():
    print(f"{currency}: {amount / rate:,.2f}")
```

تمرین ۹۶: صورت‌حساب با مالیات و انعام

صورت سؤال: کلاس ساده صورت‌حساب بنویسید که اقلام را بگیرد، ۹ درصد مالیات و ۱۰ درصد حق خدمت اضافه کند و مبلغ نهایی را با ریز اقلام چاپ کند.

```
class Invoice:
    def __init__(self):
        self.items = []

    def add_item(self, name, price):
        self.items.append((name, price))

    def total(self):
        subtotal = sum(p for _, p in self.items)
        return subtotal * 1.09 * 1.10  # 9% tax + 10% service

inv = Invoice()
inv.add_item("Pizza", 450_000)
inv.add_item("Salad", 220_000)
print(f"Final bill: {inv.total():,.0f} toman")
```

تمرین ۹۷: تحلیل نمرات کلاس

صورت سؤال: برنامه‌ای بنویسید که نمرات دانشجویان را در دیکشنری بگیرد و میانگین کلاس، نفرات برتر (نمره ۱۸ به بالا) و مردودها (کمتر از ۱۰) را گزارش کند

```
students = {"Ali": 18, "Sara": 19.5, "Reza": 9, "Mina": 12, "Omid":
20}
avg = sum(students.values()) / len(students) top = [n for n, g in
students.items() if g >= 18] failed = [n for n, g in students.items() if g
< 10] print(f"Class average: {avg:.2f}") print("Top students:", top)
print("Failed:", failed)
```

تمرین ۹۸: ژنراتور رمز تصادفی

صورت سؤال: برنامه‌ای بنویسید که طول دلخواه کاربر را بگیرد و رمزی تصادفی از حروف بزرگ و کوچک، اعداد و نمادها بسازد (با ماژول secrets برای امنیت بیشتر).

```
import secrets
import string

length = int(input("Password length: "))
alphabet = string.ascii_letters + string.digits + "!@#$%"
password = "".join(secrets.choice(alphabet) for _ in range(length))
print("Generated password:", password)
```

تمرین ۹۹: تحلیلگر متن ساده

صورت سؤال: متنی بگیرید و گزارشی از تعداد کلمات، کاراکترها (با و بدون فاصله)، پرتکرارترین کلمه و میانگین طول کلمات ارائه دهید.

```
text = input("Enter a paragraph: ")
words = text.split()
from collections import Counter
freq = Counter(words)
print(f"Words: {len(words)} | Chars: {len(text)} | No-space:
{len(text.replace(' ', ''))}")
print(f"Most common: '{freq.most_common(1)[0][0]}'")
print(f"Avg word length: {sum(map(len, words)) / len(words):.1f}")
```

تمرین ۱۰۰: مدیریت سفارش رستوران

صورت سؤال: پروژه پایانی: منوی رستوران در دیکشنری باشد؛ مشتری سفارش بدهد، فاکتور اقلام و مجموع محاسبه و در فایل JSON ذخیره شود.

```
import json
from datetime import datetime

menu = {"pizza": 450_000, "burger": 320_000, "salad": 220_000,
"juice": 90_000}
order = {}
```

```
while True:
    item = input("Item (or 'done'): ").lower()
    if item == "done":
        break
    if item in menu:
        order[item] = order.get(item, 0) + 1
    else:
        print("Not in menu!")

total = sum(menu[i] * q for i, q in order.items())
receipt = {"time": datetime.now().isoformat(), "items": order, "total":
total}
with open("orders.json", "a", encoding="utf-8") as f:
    f.write(json.dumps(receipt, ensure_ascii=False) + "\n")
print(f"Order saved. Total: {total:,} toman")
```

پایان مجموعه — اگر این ۱۰۰ تمرین را با تمرکز حل کرده باشید، اکنون با اکثر ابزارهای روزمره پایتون آشنا هستید؛ قدم بعدی، ساخت پروژه‌های واقعی خودتان است. موفق باشید!